

# Three.js Code

Three.js ek JavaScript library hai jo 3D graphics ke liye use hoti hai - isi wajah se ye standalone file nahi hai, JavaScript ke andar hi call hoti hai. Neeche wahi parts hain jahan Three.js (scene, camera, lighting, 3D shapes, raycaster) use hui hai, alag-alag pieces mein.

## Scene, Camera, Renderer Setup

```
// ----- basic setup -----
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x0b1622);
scene.fog = new THREE.Fog(0x0b1622, 20, 75);

const camera = new THREE.PerspectiveCamera(75, window.innerWidth/window.innerHeight, 0.1, 1000);
camera.position.set(0, 1.7, 0);

const renderer = new THREE.WebGLRenderer({antialias:true});
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio,2));
renderer.shadowMap.enabled = true;
document.body.appendChild(renderer.domElement);

window.addEventListener('resize', ()=>{
  camera.aspect = window.innerWidth/window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

## Lighting (Ambient + Directional Light)

```
// ----- lights -----
scene.add(new THREE.AmbientLight(0x8899aa, 0.7));
const sun = new THREE.DirectionalLight(0xffffff, 0.9);
sun.position.set(30,40,10);
sun.castShadow = true;
sun.shadow.mapSize.set(1024,1024);
sun.shadow.camera.left=-40; sun.shadow.camera.right=40;
sun.shadow.camera.top=40; sun.shadow.camera.bottom=-40;
scene.add(sun);
```

## Arena: Ground, Walls, Obstacles (Geometry + Materials)

```
// ----- arena -----
const ARENA = 34;
const groundGeo = new THREE.PlaneGeometry(ARENA*2, ARENA*2, 10, 10);
const groundMat = new THREE.MeshStandardMaterial({color:0x2c3e40, roughness:0.95});
const ground = new THREE.Mesh(groundGeo, groundMat);
ground.rotation.x = -Math.PI/2;
ground.receiveShadow = true;
scene.add(ground);

const wallMat = new THREE.MeshStandardMaterial({color:0x1a2530, roughness:0.9});
function addWall(x,z,w,d,h){
  const wall = new THREE.Mesh(new THREE.BoxGeometry(w,h,d), wallMat);
  wall.position.set(x,h/2,z);
  wall.castShadow = true; wall.receiveShadow = true;
  scene.add(wall);
}
addWall(0,-ARENA,ARENA*2,1,6);
addWall(0,ARENA,ARENA*2,1,6);
addWall(-ARENA,0,1,ARENA*2,6);
addWall(ARENA,0,1,ARENA*2,6);

const coverMat = new THREE.MeshStandardMaterial({color:0x4a5a52, roughness:0.85});
const obstacles = [];
function addBox(x,z,sx,sy,sz){
  const b = new THREE.Mesh(new THREE.BoxGeometry(sx,sy,sz), coverMat);
  b.position.set(x, sy/2, z);
  b.castShadow = true; b.receiveShadow = true;
```

```

    scene.add(b);
    obstacles.push({x:x, z:z, hw:sx/2, hd:sz/2});
}
const layout = [
    [8,8,4,2.2,4],[ -8,8,3,2.6,3],[8, -8,4,2,4],[ -8, -8,3,2.4,5],
    [0,14,6,2,2],[0, -14,6,2,2],[14,0,2,2,6],[ -14,0,2,2,6],
    [4,0,2,1.6,2],[ -5,3,2,1.8,2],[18,18,3,2,3],[ -18, -18,3,2,3]
];
layout.forEach(a=>addBox(...a));

```

## Enemies: 3D Mesh Creation (CapsuleGeometry)

```

// ----- enemies -----
const enemies = [];
const enemyGeo = new THREE.CapsuleGeometry(0.45, 1.1, 4, 8);
function spawnEnemy(){
    const mat = new THREE.MeshStandardMaterial({color:0xc0392b, roughness:0.7});
    const mesh = new THREE.Mesh(enemyGeo, mat);
    mesh.castShadow = true;
    let x,z;
    do{
        const ang = Math.random()*Math.PI*2;
        const r = 15 + Math.random()*15;
        x = Math.cos(ang)*r; z = Math.sin(ang)*r;
    } while(Math.abs(x)>ARENA-2 || Math.abs(z)>ARENA-2);
    mesh.position.set(x, 1.05, z);
    scene.add(mesh);
    const enemy = { mesh, health:60, speed: 1.3+Math.random()*0.6+wave*0.06, alive:true, attackCd:0
    };
    mesh.userData.enemyRef = enemy;
    enemies.push(enemy);
}

function damageEnemy(enemy, dmg){
    if(!enemy.alive) return;
    enemy.health -= dmg;
    enemy.mesh.material.emissive = new THREE.Color(0x330000);
    setTimeout(()=>{ if(enemy.mesh.material) enemy.mesh.material.emissive = new
        THREE.Color(0x000000); },80);
    if(enemy.health<=0){
        enemy.alive = false;
        scene.remove(enemy.mesh);
        score += 100;
        kills++;
        updateHUD();
        checkWave();
    }
}

function checkWave(){
    if(kills >= killsNeededForWave){
        wave++; kills = 0; killsNeededForWave = 5 + wave*2;
        document.getElementById('waveVal').textContent = wave;
        const toSpawn = 3 + wave;
        for(let i=0;i<toSpawn;i++) setTimeout(spawnEnemy, i*250);
    }
}
for(let i=0;i<4;i++) spawnEnemy();

```

## Loot Items: 3D Mesh Creation (OctahedronGeometry)

```

function spawnLootAt(idx){
    const [x,z] = lootPoints[idx];
    const type = LOOT_TYPES[Math.floor(Math.random()*LOOT_TYPES.length)];
    let color, weaponKey=null;
    if(type==='ammo') color=0xffd23f;
    else if(type==='health') color=0x4caf50;
    else { const keys=['SMG','Shotgun','Sniper']; weaponKey =
        keys[Math.floor(Math.random()*keys.length)]; color = WEAPONS[weaponKey].color; }
    const geo = new THREE.OctahedronGeometry(0.4,0);
    const mat = new THREE.MeshStandardMaterial({color, emissive:color, emissiveIntensity:0.35,
        roughness:0.4});
    const mesh = new THREE.Mesh(geo, mat);
    mesh.position.set(x, 1.1, z);
    mesh.castShadow = true;
    scene.add(mesh);
    loot[idx] = { mesh, x, z, type, weaponKey, active:true, bob: Math.random()*Math.PI*2 };
}

```

```
}
```

## Shooting: Raycaster (hit detection)

```
const raycaster = new THREE.Raycaster();
const muzzleEl = document.getElementById('muzzle');
const hitFlashEl = document.getElementById('hitFlash');

canvas.addEventListener('mousedown', (e)=>{
  if(!gameActive || isTouch || e.button!==0) return;
  shoot();
});

function switchWeapon(key){
  if(currentWeaponKey===key || reloading) return;
  currentWeaponKey = key;
  ammo = WEAPONS[key].magSize;
  updateHUD();
}

function tryReload(){
  if(reloading || !player.alive) return;
  const w = WEAPONS[currentWeaponKey];
  if(ammo===w.magSize || reserve[currentWeaponKey]<=0) return;
  reloading = true;
  document.getElementById('reloadHint').textContent = 'Reloading...';
  setTimeout(()=>{
    const need = w.magSize - ammo;
    const take = Math.min(need, reserve[currentWeaponKey]);
    ammo += take;
    reserve[currentWeaponKey] -= take;
    reloading = false;
    updateHUD();
  }, 1000);
}

function shoot(){
  if(!gameActive || !player.alive || reloading) return;
  const w = WEAPONS[currentWeaponKey];
  const now = performance.now();
  if(now - lastShotTime < w.cooldown) return;
  if(ammo<=0){ tryReload(); return; }
  lastShotTime = now;
  ammo--;
  updateHUD();
  muzzleEl.style.opacity = 1;
  setTimeout(()=>muzzleEl.style.opacity=0, 60);

  const pellets = w.pellets || 1;
  const meshes = enemies.map(en=>en.mesh);
  for(let p=0;p<pellets;p++){
    const sx = (Math.random()-0.5)*w.spread*2;
    const sy = (Math.random()-0.5)*w.spread*2;
    raycaster.setFromCamera({x:sx,y:sy}, camera);
    const hits = raycaster.intersectObjects(meshes, true);
    if(hits.length>0){
      let obj = hits[0].object;
      while(obj.parent && !obj.userData.enemyRef) obj = obj.parent;
      const enemy = obj.userData.enemyRef;
      if(enemy) damageEnemy(enemy, w.damage);
    }
  }
}
```

## Render Loop: Camera Update + Scene Render

```
const clock = new THREE.Clock();
function animate(){
  requestAnimationFrame(animate);
  const dt = Math.min(clock.getDelta(), 0.05);

  if(gameActive && player.alive){
    const forward = new THREE.Vector3(Math.sin(player.yaw), 0, Math.cos(player.yaw)).negate();
    const right = new THREE.Vector3(Math.cos(player.yaw), 0, -Math.sin(player.yaw));
    let move = new THREE.Vector3();

    if(isTouch){
```

```

    if(Math.abs(joyVec.x)>0.08 || Math.abs(joyVec.y)>0.08){
        move.add(forward.clone().multiplyScalar(-joyVec.y));
        move.add(right.clone().multiplyScalar(joyVec.x));
    }
} else {
    if(keys['KeyW']) move.add(forward);
    if(keys['KeyS']) move.sub(forward);
    if(keys['KeyD']) move.add(right);
    if(keys['KeyA']) move.sub(right);
}

if(move.lengthSq()>0){
    move.normalize();
    const spd = player.speed * ((!isTouch && (keys['ShiftLeft']||keys['ShiftRight'])) ?
        player.sprintMult : 1);
    const nx = player.pos.x + move.x*spd*dt;
    const nz = player.pos.z + move.z*spd*dt;
    const clampedX = Math.max(-ARENA+1, Math.min(ARENA-1, nx));
    const clampedZ = Math.max(-ARENA+1, Math.min(ARENA-1, nz));
    if(!collidesWithObstacle(clampedX, player.pos.z, 0.5)) player.pos.x = clampedX;
    if(!collidesWithObstacle(player.pos.x, clampedZ, 0.5)) player.pos.z = clampedZ;
}
camera.position.set(player.pos.x, player.height, player.pos.z);
const dir = new THREE.Vector3(
    Math.sin(player.yaw)*Math.cos(player.pitch),
    Math.sin(player.pitch),
    -Math.cos(player.yaw)*Math.cos(player.pitch)
);
camera.lookAt(camera.position.clone().add(dir));

enemies.forEach(en=>{
    if(!en.alive) return;
    const ex=en.mesh.position.x, ez=en.mesh.position.z;
    const dx = player.pos.x - ex, dz = player.pos.z - ez;
    const dist = Math.sqrt(dx*dx+dz*dz);
    if(dist > 1.6){
        const nx = ex + (dx/dist)*en.speed*dt;
        const nz = ez + (dz/dist)*en.speed*dt;
        en.mesh.position.x = nx;
        en.mesh.position.z = nz;
        en.mesh.rotation.y = Math.atan2(dx,dz);
    } else {
        en.attackCd -= dt;
        if(en.attackCd<=0){
            en.attackCd = 0.8;
            player.health -= 8;
            flashDamage();
            updateHUD();
            if(player.health<=0) gameOver();
        }
    }
});

loot.forEach(item=>{
    if(!item || !item.active) return;
    item.bob += dt*2.2;
    item.mesh.position.y = 1.1 + Math.sin(item.bob)*0.12;
    item.mesh.rotation.y += dt*1.4;
    const dx = player.pos.x-item.x, dz = player.pos.z-item.z;
    if(Math.sqrt(dx*dx+dz*dz) < 1.4) collectLoot(item);
});

drawMinimap();
}

renderer.render(scene, camera);
}

```